

Peretto

marcos
Luciano

PDI — ATIVIDADE 03 · REGISTRO DE ENTREGA

Mapeamento de Domínios com Domain-Driven Design (DDD)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company — Automação & Infraestrutura

Módulo: 02 · Práticas de Engenharia e Clean Code

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

bounded contexts / agregados / eventos

1. Contexto

A FV opera em **múltiplos domínios**: Imobiliário, Saúde, Varejo e Serviços. Hoje o time modela tudo num único modelo de dados gigante, onde alterar um campo de imóvel quebra a regra de agendamento de saúde. O domínio virou um emaranhado onde **todo conceito conhece todo conceito**.

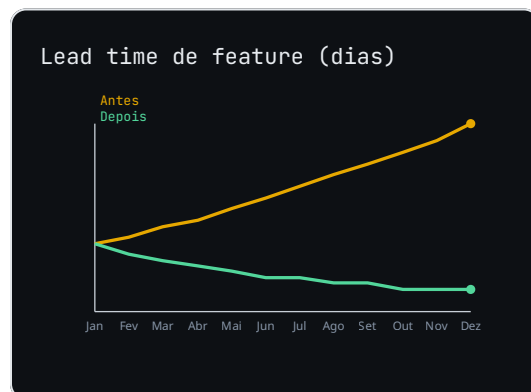
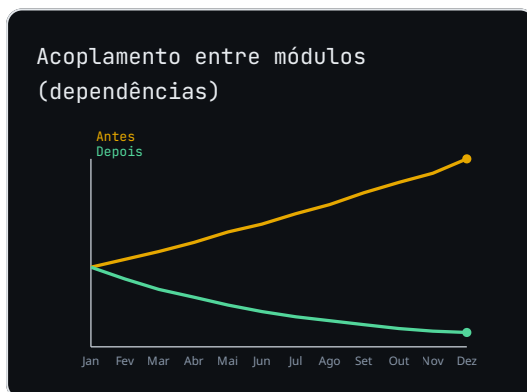
ANALOGIA DO PRÉDIO DE APARTAMENTOS

Um condomínio tem moradores, elevadores, portaria e academia. Cada um é um contexto delimitado: a portaria não precisa saber o que você come na cozinha, e a academia não manda na sua conta de luz. Se tentássemos um único manual para tudo, ninguém

entenderia nada. DDD é organizar o prédio em **áreas com regras próprias**, que conversam por recados (eventos).

2. Diagnóstico

Sem fronteiras de domínio, mudanças têm efeito cascata e as regras de negócio ficam espalhadas em ifs por todo o código. O tempo de impacto de uma alteração é imprevisível e a linguagem do time diverge da linguagem do código.



Raiz do problema: um único modelo onipresente força regras de domínios distintos a conviverem, destruindo a linguagem ubíqua e criando acoplamento oculto.

3. Solução

Adotar **DDD (Domain-Driven Design)** dividindo o sistema em **bounded contexts** (contextos delimitados), cada um com sua **linguagem ubíqua** — o vocabulário que negócio e código compartilham sem tradução. Dentro de cada contexto modelamos **agregados** (raiz + entidades + value objects com consistência transacional) e comunicamos mudanças via **eventos de domínio**.

Mapeamento dos contextos da FV:

```
# Contextos delimitados
Imobiliario -> Agregado Raiz: Imovel      (Proposta, Visita)
Saude       -> Agregado Raiz: Paciente    (Agendamento, Prontuario)
```

Varejo	-> Agregado Raiz: Pedido	(Item, Pagamento)
Servicos	-> Agregado Raiz: Ordem	(Execucao, NotaFiscal)

Agregado garante consistência: só a raiz é referenciada de fora; filhos não têm identidade pública:

```
class Pedido:                                # Aggregate Root
    def __init__(self, id):
        self.id = id
        self.itens = []                      # entidades internas
        self._eventos = []
    def adicionar(self, produto, qtd):
        self.itens.append(Item(produto, qtd))
        self._eventos.append(PedidoAtualizado(self.id))
```

Eventos de domínio desacoplam os contextos — Saúde não chama Imobiliário direto, apenas publica:

```
@dataclass
class PedidoPago(DomainEvent):
    pedido_id: str
    total: float

# orquestrador publica; outros contextos reagem
bus.publish(PedidoPago(pedido_id, total))
```

4. Como funciona (pipeline)

Modelagem DDD da FV

| FLUXO EM EXECUÇÃO



5. Antes vs Depois

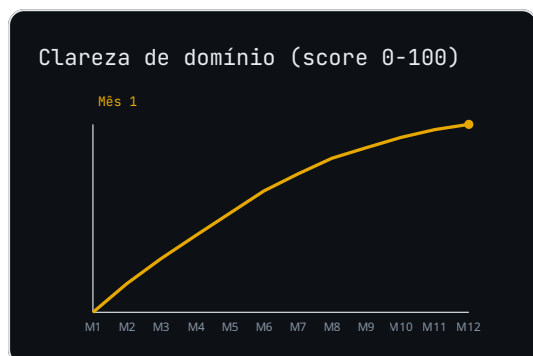
CENÁRIO	ANTES (MODELO ÚNICO)	DEPOIS (DDD)
Mudar regra de Saúde	Risco no Imobiliário	Isolado no contexto

Vocabulário	Negócio $\neq$ código	Linguagem ubíqua
Consistência	Ad-hoc	Agregado transacional
Integração	SQL cruzado	Eventos de domínio

6. Entregas

- **Mapa** `bounded-contexts.svg`: fronteiras da FV e fluxo de eventos.
- **Glossário** `ubiquitous-language.md` por contexto.
- **Módulos** `imobiliario/`, `saude/`, `varejo/`, `servicos/` com agregados.
- **Eventos** `domain_events.py`: contratos versionados.

7. Métricas



Meta: 100% dos contextos com glossário ubíquo documentado e eventos versionados entre eles.

8. Status final

NÃO PUBLICADO — Desenvolvido e em homologação. Aguarda revisão antes de produção.